

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 9

JavaScript im Browser

9.1 Objekte

JavaScript ist eine objektorientierte Sprache. Da Objekte eine Voraussetzung für die Arbeit mit JavaScript im Browser sind, soll hier zunächst eine kurze Einführung in die Thematik gegeben werden, um ein grundsätzliches Verständnis dafür aufzubauen. Es werden bewusst nicht alle Details und Hintergründe erläutert, da dies in der Lehrveranstaltung *Algorithmisches Denken* geschieht. Ziel in *Web Fundamentals* ist es, dass sie mit Objekten grundsätzlich arbeiten können und ihren Sinn bzw. grundsätzliche Funktionsweise verstehen.

9.1.1 Was ist ein Objekt?

Objekte sind eine weitere Art von Datentyp, zusätzlich zu den in Abschnitt 8.3 erwähnten. Im Gegensatz zu diesen (Arrays ausgenommen) enthalten sie nicht einfach nur einen einzigen Wert, sondern kapseln eine Vielzahl von Daten, die auch thematisch zusammengehören. Sie zählen daher zu den komplexen Datentypen.

Während primitive Datentypen lediglich einen Wert enthalten, bilden Objekte komplexere Inhalte ab die sich an Dingen aus dem realen Leben orientieren. Ein Beispiel:

In JavaScript sollen die Daten und das Verhalten von Studierenden erfasst bzw. abgebildet werden. Erfasst werden sollen Name, Alter, Studiengang und die beherrschten Programmiersprachen. Zudem sollen die beherrschten Programmiersprachen ausgegeben und auch neue erlernt, d. h. hinzugefügt werden können. Mit einfachen, primitiven Datentypen und Funktionen könnte dies so aussehen:

```
1 // Daten des*der Studenten*Studentin
2 let studentName = "John Doe";
3 let studentAge = 22;
4 let studentProgram = "MTD";
5 let studentLanguages = ["Java", "C#"];
6
7 // Ausgabe bzw. Änderung der Daten
8 function showLanguages() {
9   console.log(`${studentName} (${studentAge}, ${studentProgram}) beherrscht diese
      Sprachen: ${studentLanguages}`);
10 }
11
```

```

12 function learnNewLanguage(language) {
13     studentLanguages[studentLanguages.length] = language;
14 }
15
16 // Aufrufe
17 showLanguages(); // Java,C#
18 learnNewLanguage("JavaScript");
19 showLanguages(); // Java,C#,JavaScript

```

Während dies zwar funktioniert, haben die einzelnen Variablen und Funktionen logisch gesehen nichts miteinander zu tun. Vielmehr existieren Sie als globale Variablen und Funktionen im gesamten Script, dass `showLanguages()` etwa mit den Werten von `studentName`, `studentAge`, `studentProgram` oder `studentLanguages` arbeitet ist zwar möglich und auch sinnvoll, müsste aber nicht so sein. Es könnte sich dabei auch um vier Variablen und zwei Funktionen handeln, die völlig unterschiedliche Zwecke haben, da es nichts gibt, dass sie logisch miteinander verbindet. Objekte handhaben dies anders.

Objekte: Datenkapseln mit Eigenschaften und Methoden

Ein Objekt stellt eine solche logische Klammer dar. Es kapselt Daten, die inhaltlich zusammengehören. Dabei bestehen Objekte aus zwei Dingen: *Eigenschaften* (engl. „properties“), auch Attribute genannt, repräsentieren die Daten bzw. den Zustand des Objekts. *Methoden* definieren das Verhalten des Objekts. Sie sind nichts anderes als Funktionen, die allerdings zum Objekt gehören und dessen Daten, d.h. die Eigenschaften ausgeben oder verändern.

Das Studierendenbeispiel als Objekt konstruiert sieht nun wie folgt aus:

```

1 // Daten des*der Studenten*Studentin & Ausgabe bzw. Änderung der Daten
2 const student = {
3     name: "John Doe",
4     age: 22,
5     program: "MTD",
6     languages: ["Java", "C#"],
7     showLanguages() {
8         console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese
          Sprachen: ${this.languages}`);
9     },
10    learnNewLanguage(language) {
11        this.languages[this.languages.length] = language;
12    }
13 };
14
15 // Aufrufe
16 student.showLanguages(); // Java,C#
17 student.learnNewLanguage("JavaScript");
18 student.showLanguages(); // Java,C#,JavaScript

```

Ersichtlich ist, dass nun alles – sowohl die Daten des*der Studenten*Studentin, als auch die Funktionen zum Ausgeben und Erlernen von Sprachen – in einer einzigen Variable, nämlich `student` gespeichert sind. Diese Variable enthält das Objekt, dessen Bestandteile in den geschweiften Klammern notiert sind.

Die Bezeichner `name`, `age`, `program`, `languages`, `showLanguages` und schließlich `learnNewLanguage` fungieren dabei als die Schlüssel (Keys), über die auf die Inhalte

zugegriffen wird.

Dabei kommen zwei Schreibweisen zum Einsatz:

- **Eigenschaften (Daten):** Handelt es sich beim Wert um einen Datentyp (String, Zahl, Array etc.), so wird das Schema **Schlüssel: Wert** verwendet. `name`, `age`, `program` und `languages` sind solche Eigenschaften.
- **Methoden (Verhalten):** Handelt es sich um eine Funktion, so wird die moderne Kurzschreibweise (Method Shorthand) verwendet. Sie ähnelt der Methodendefinition in Java: Der Schlüssel wird wie ein Funktionsname notiert, gefolgt von Klammern und dem Funktionsrumpf. Das Schlüsselwort `function` und der Doppelpunkt entfallen hier. `showLanguages` und `learnNewLanguage` sind die Methoden dieses Objekts.

Diese Variable, d. h. dieses Objekt, repräsentiert eine*n Studentin*Studenten als logische Einheit. Dies zeigt sich auch im Zugriff: Um die Methode `showLanguages()` auszuführen, muss sie über die angelegte Objekt-Variable aufgerufen werden – also mittels `student.showLanguages()`. Eine globale Funktion `showLanguages()`, die für sich allein steht, existiert in diesem Kontext nicht.

9.1.2 Zugriff auf Eigenschaften und Methoden

Der *Zugriff* auf einzelne Eigenschaften oder Methoden eines Objekts erfolgt immer über eine simple Punktnotation:

```
1 <Name des Objekts>.<Name der Eigenschaft>
2 <Name des Objekts>.<Name der Methode()>
```

Dies war schon in Abschnitt 8.7 zu sehen, als der Aufruf `console.log()` (Output in die Konsole der Developer-Tools schreiben) verwendet wurde. `console` ist ein Objekt, `log()` ist eine Methode dieses Objekts.

Im Studierenden-Beispiel ist `student` das Objekt, der Aufruf zum Ausgeben der beherrschten Sprachen sowie zum Hinzufügen einer neuen Sprache wird wie folgt formuliert:

```
1 student.showLanguages();
2 student.learnNewLanguage("JavaScript");
```

Die erste Methode ist rein zur Ausgabe und manipuliert das Objekt nicht. Die zweite hingegen verändert die Eigenschaft `language`, indem sie einen neuen Eintrag zum Array hinzufügt.

Die Punktnotation gilt auch für Eigenschaften. Will man etwa den Namen des*der Studenten*Studentin abfragen und in der Konsole ausgeben, sowie dieser Person nach dem Geburtstag ein neues Alter setzen, wird dies wie folgt geschrieben:

```
1 console.log(student.name);
2 student.age = 23;
```

9.1.3 Das eigene Objekt mit `this` ansprechen

Bei den Objektmethoden `showLanguages()` und `learnNewLanguage()` fällt auf, dass der Zugriff auf die Eigenschaften für Name, Alter, Studiengang und beherrschten Sprachen nicht einfach über ihren direkten Eigenschaftsnamen geschieht. Vielmehr erfolgt der Zugriff über `this.name`, `this.age`, `this.program` oder `this.languages`.

Das Schlüsselwort `this` repräsentiert – wenn es wie hier innerhalb eines Objekts verwendet wird – immer das Objekt selbst. Auf den konkreten Fall umgelegt, steckt in `this` also das gesamte Studierenden-Objekt, also äquivalent zu `student`. Letzteres ist allerdings nur der Variablenname des Objekts, der ja beliebig gewählt ist und sich jederzeit ändern könnte. Somit wäre es nicht zielführend, `student.name` etc. zu schreiben, denn würde die Variable anders gewählt, würde dies nicht mehr funktionieren.

Mit der Angabe von `this.name` wird also explizit angegeben, dass die Eigenschaft `name` des aktuellen Objekts ausgegeben (oder verändert) werden soll. Würde in der Methode nicht `this.name` sondern nur `name` verwendet, so würde versucht werden, auf eine globale oder auch lokale Variable `name` zuzugreifen.

`this` und der Kontext

`this` verweist immer auf das Objekt, in dessen Kontext es gerade verwendet wird. Wird es in einer Objektmethode verwendet, so enthält es das Objekt, zu dem die Methode gehört.

Außerhalb von Objekten existiert `this` auch. Außerhalb jeglicher Funktion, also im globalen Kontext, zeigt `this` immer auf das globale `window`-Objekt. Innerhalb von Funktionen zeigt es im normalen Modus ebenfalls auf das globale `window`-Objekt, im strikten Modus enthält `this` allerdings `undefined`, wenn nicht explizit ein Wert gesetzt wird oder die Funktion etwa über das `window`-Objekt aufgerufen wird.

9.1.4 Definieren von Objekten

Die in Abschnitt 9.1.1 demonstrierte Art, ein Objekt zu definieren, nennt sich Objekt-Literal-Syntax. Sie stellt jedoch nur eine von mehreren Arten dar. Es existieren ebenfalls Konstruktorfunktionen und seit ECMAScript 2015 auch Klassen.

Objekt-Literal-Schreibweise

Die *Objekt-Literal-Schreibweise* ist die einfachste Art und Weise, ein Objekt zu definieren. Nach der Angabe eines Variablennamens werden die Daten des Objekts in geschwungene Klammern geschrieben. Darin werden, durch Beistriche getrennt, Eigenschaften (als **Schlüssel: Wert**) und Methoden (in der Kurzschreibweise) notiert:

```
1 const myObject = {  
2   key1: value1,  
3   key2: value2,  
4   method1() { ... }  
5 };
```

Da es sich um die Definition einer Variable handelt (auch wenn diese über mehrere Zeilen geht), sollte auf den abschließenden Strichpunkt nach der geschlossenen geschwungenen Klammer nicht vergessen werden.

Die Objekt-Literal-Schreibweise ist die schnellste und einfachste Art, in JavaScript Objekte zu erzeugen. Sie eignet sich vor allem für Objekte gut, die primär aus Eigenschaften bestehen, wenngleich die Definition von Methoden auch problemlos möglich ist.

Der Nachteil dieser Syntax ist jedoch, dass hier eine spezifische Instanz eines Objekts direkt erzeugt wird. Wird ein zweites Objekt benötigt, das identisch aufgebaut ist, aber andere Daten enthalten soll, so muss es erneut komplett definiert, d. h. notiert werden.

Dies ist vor allem bei Methoden unerwünscht, weil hier Codeverdoppelung entsteht, da die Methoden ja nur die Namen der Eigenschaften verwenden, also komplett gleich formuliert werden.

Konstruktorfunktionen

Eine *Konstruktorfunktion* behebt das Problem der einzigartigen Objekte, das die Objekt-Literal-Schreibweise mit sich bringt. Sie ist eine Funktion, die zum Erzeugen neuer Objekte verwendet werden kann. Da Konstruktorfunktionen wie reguläre Funktionen aussehen, wird in der Regel Pascal Case, also ein großer Anfangsbuchstabe mit Großbuchstaben an Wortgrenzen verwendet, um sie zu unterscheiden.

Eine Konstruktorfunktion enthält üblicherweise Parameter, die zum Initialisieren der Objekteigenschaften verwendet werden. Die Syntax sieht wie folgt aus:

```
1 function MyConstructorFunction(param1, param2) {  
2   this.key1 = param1;  
3   this.key2 = param2;  
4   this.key3 = function() { ... }  
5 }
```

Die Objekteigenschaften erhalten wieder Namen, die über das `this`-Schlüsselwort notiert werden. Als Werte werden meist die Argumente der Parameter zugewiesen. Zusätzlich können auch Methoden definiert werden, die in diesem Fall Funktionen sind, die weiteren Eigenschaften zugewiesen werden (eine Kurzschreibweise wie beider Objekt-Literal-Syntax existiert hier nicht).

Das Studierendenbeispiel realisiert als Konstruktorfunktion definiert sich folgendermaßen:

```
1 function Student(name, age, program, languages) {  
2   this.name = name;  
3   this.age = age;  
4   this.program = program;  
5   this.languages = languages;  
6   this.showLanguages = function () {  
7     console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese  
8       Sprachen: ${this.languages}`);  
9   };  
10  this.learnNewLanguage = function (language) {  
11    this.languages[this.languages.length] = language;  
12  }  
13 }
```

Die Funktion heißt `Student` und hat vier Parameter, die 1:1 den Eigenschaften `name`, `age`, `program` und `languages` zugewiesen werden. Weiters werden die zwei bekannten Methoden `showLanguages()` und `learnNewLanguage()` definiert, die mit den Eigenschaften arbeiten.

Dies ist jedoch nur die *Definition* der Konstruktorfunktion. Durch sie alleine wird noch kein Objekt erzeugt. Dieser Vorgang geschieht mit dem Schlüsselwort `new` und dem Namen der Konstruktorfunktion:

```
1 const myObject1 = new MyConstructorFunction(arg1, arg2);  
2 const myObject2 = new MyConstructorFunction(arg3, arg4);
```

Das Schlüsselwort **new** macht die an sich „normale“ Funktion zur Konstruktorfunktion. Ersichtlich ist, dass keinerlei **return**-Anweisung darin vorkommt, stattdessen erzeugt die Funktion in Kombination mit **new** implizit ein neues Objekt und gibt es zurück, sodass es einer Variable zugewiesen werden kann. Ebenso ist es nun möglich, mehrere Objekte desselben Typs zu erzeugen, in dem die Konstruktorfunktion mehrmals mit unterschiedlichen Argumenten aufgerufen wird.

Umgelegt auf das Studierendenbeispiel können nun beliebig viele (in diesem Fall zwei) Studierende erzeugt werden. Ihre Struktur, d. h. ihre Eigenschaften und Methoden, ist identisch. Die Werte der Eigenschaften unterscheiden sich jedoch.

```
1 const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);
2 const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);
3
4 student1.showLanguages(); // Java,C#
5 student1.learnNewLanguage("JavaScript");
6 student1.showLanguages(); // Java,C#,JavaScript
7 student2.showLanguages(); // Pascal,C,Java
```

Hinweis zur Performance

Werden Objekte auf diese Weise erstellt, wird für jedes neue Objekt (jede*n Studierende*n) auch jede Methode im Arbeitsspeicher neu angelegt. Bei drei Studierenden existiert die Funktion **showLanguages** also dreimal im Speicher.

In älteren JavaScript-Versionen wurde dies durch komplizierte Zuweisungen an den sogenannten **prototype** gelöst. In modernem JavaScript gibt es dafür eine wesentlich elegantere Lösung, die dieses Problem im Hintergrund automatisch behebt: **Klassen**.

Klassen

Seit ECMAScript 2015 gibt es auch die Möglichkeit, Objekte über die Angabe einer *Klasse* zu definieren. Klassen sind in anderen Programmiersprachen quasi die Baupläne für Objekte. Sie bestehen in der Regel aus einem Konstruktor, der implizit beim Anlegen eines Objekts mit **new** aufgerufen wird und in dem die Zuweisungen der Eigenschaften geschehen und der Definition der Methoden.

Die Klassensyntax in JavaScript imitiert dieses Verhalten. Technisch gesehen basiert sie im Hintergrund weiterhin auf dem Prinzip der Konstruktorfunktionen und Prototypen, weshalb sie oft als syntaktische Alternative („Syntactic Sugar“) bezeichnet wird. Allerdings bietet die **class**-Syntax einen entscheidenden Vorteil: Methoden, die innerhalb der Klasse definiert werden, werden automatisch speicherschonend angelegt (sie landen im sogenannten Prototyp) und nicht für jedes Objekt einzeln kopiert.

Die Syntax lautet wie folgt: mit dem Schlüsselwort **class** wird der Name der „Klasse“ spezifiziert, alle weiteren Angaben folgen in geschweiften Klammern. Der Konstruktor muss immer **constructor()** heißen und wird wie eine Funktion, allerdings ohne das Schlüsselwort **function** angegeben. Alle Methoden werden im Anschluss spezifiziert, das Schlüsselwort **function** entfällt ebenfalls wieder.

```
1 class MyClass {
2   constructor(param1, param2) {
3     this.key1 = param1;
4     this.key2 = param2;
```

```

5   }
6   method1() { ... }
7 }

```

Umgelegt auf das Studierendenbeispiel ergibt sich die folgende Definition:

```

1 class Student {
2   constructor(name, age, program, languages) {
3     this.name = name;
4     this.age = age;
5     this.program = program;
6     this.languages = languages;
7   }
8
9   showLanguages() {
10    console.log(`${this.name} (${this.age}, ${this.program}) beherrscht diese
        Sprachen: ${this.languages}`);
11  }
12
13  learnNewLanguage(language) {
14    this.languages[this.languages.length] = language;
15  }
16 }

```

Das Erzeugen eines neuen Studierenden-Objekts erfolgt genauso wie bei den Konstrukturfunktionen, der Zugriff auf Methoden ist ebenfalls gleich und wird nicht mehr angeführt:

```

1 const student1 = new Student("John Doe", 22, "MTD", ["Java", "C#"]);
2 const student2 = new Student("Jane Doe", 25, "SE", ["Pascal", "C", "Java"]);

```

9.1.5 Vordefinierte Objekte

JavaScript (ECMAScript) bietet eine Reihe von vordefinierten Objekten, die nützliche Funktionalitäten zur Verfügung stellen. Eine vollständige Aufzählung bieten [2, Kapitel 20–28] oder MDN¹, hier werden zunächst einige wichtige überblicksartig beschrieben.

Object

Von diesem Objekt stammen alle anderen Objekte in JavaScript ab, somit ist jedes Objekt – egal ob selbst erstellt oder vordefiniert – immer auch von Typ `Object`.

Ein neues Objekt kann auch wie folgt erzeugt werden, dies ist aber eher ungebräuchlich und in der Regel wird die Objekt-Literal-Syntax stattdessen verwendet.

```

1 const data = new Object();

```

Wesentlich häufiger kommen jedoch die statischen Hilfsmethoden zum Einsatz, um Inhalte eines Objekts als Array auszulesen (etwa um danach mit Array-Methoden weiterzuarbeiten):

- `Object.keys(obj)`: liefert ein Array aller Eigenschaftsnamen (Schlüssel) zurück.
- `Object.values(obj)`: liefert ein Array aller Werte zurück.

¹https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

- `Object.entries(obj)`: liefert ein Array aus Arrays zurück, wobei jedes innere Array ein Paar aus [Schlüssel, Wert] enthält.

```
1 const person = { name: "John", age: 22 };
2 console.log(Object.keys(person)); // ["name", "age"]
3 console.log(Object.values(person)); // ["John", 22]
4 console.log(Object.entries(person)); // [["name", "John"], ["age", 22]]
```

Number

Number ist die Objektversion für alle primitiven Zahlendatentypen. Diese existiert, um mit Zahlen bestimmte Operationen (z. B. Rundung oder Umwandlung) durchführen zu können bzw. um Informationen über bestimmte Zahlenwerte im System zu bekommen.

Entscheidend für Objektversionen von primitiven Datentypen ist, dass sie nicht explizit mit `new` angelegt werden müssen. Es ist also unüblich, eine Zahl in ein **Number**-Objekt zu verpacken, da dies eher unübersichtlich und auch langsamer ist.

Dennoch ist Folgendes möglich: zunächst wird ein **Number**-Objekt angelegt und dann mit der Methode `toFixed()`, die jedes **Number**-Objekt besitzt, auf zwei Nachkommastellen gerundet und dies ausgegeben.

```
1 const number1 = new Number(12.3456);
2 const number1rounded = number1.toFixed(2);
3 console.log(number1rounded); // 12.35
```

Diese Methoden können jedoch auch direkt mit Zahlen, d. h. mit den primitiven Datentypen verwendet werden. JavaScript konvertiert dieses im Hintergrund automatisch in das entsprechende **Number**-Objekt, der Code bleibt jedoch übersichtlich. Das obige Beispiel sollte also immer wie folgt geschrieben werden:

```
1 const number2 = 12.3456;
2 const number2rounded = number2.toFixed(2);
3 console.log(number2rounded); // 12.35
```

Im **Number**-Objekt befinden sich auch diverse statische Eigenschaften, die abgefragt werden können. Dazu werden der Klassenname und die Eigenschaft in der Punktnotation verwendet. Hier werden die größte und kleinste darstellbare Zahl im System abgefragt und ausgegeben:

```
1 console.log(Number.MAX_VALUE); // größte darstellbare Zahl
2 console.log(Number.MIN_VALUE); // kleinste darstellbare Zahl
```

String

Auch Zeichenketten sind Objekte, nämlich vom Typ **String**. Sie könnten ebenso über `const text = new String("Hallo Welt");` erzeugt werden, was aber wiederum unüblich ist. Vielmehr bietet das **String**-Objekt wieder viele nützliche Methoden, darunter die folgenden:

- `charAt(n)`: gibt das Zeichen an der Position *n* zurück (alternativ ist der Zugriff auch wie bei Arrays über `string[n]` möglich),
- `indexOf(char)`: gibt die Position des ersten Vorkommens von `char` zurück (oder `-1`, falls nicht gefunden),

- `includes(char)`: gibt `true` zurück, wenn der String `char` enthält (moderne Alternative zu `indexOf`, wenn die Position egal ist),
- `substring(start, end)`: gibt den Teilstring zwischen den Indizes `start` und `end` (exklusive) zurück,
- `slice(start, end)`: ähnlich wie `substring`, akzeptiert jedoch auch negative Werte, um vom Ende des Strings zu zählen,
- `split(char)`: teilt den String bei allen Vorkommnissen von `char` in ein Array auf.

Die Eigenschaft `length` gibt die Anzahl der Zeichen im String zurück.

```

1  const text = "Das ist ein langer Text";
2
3  // Zeichen an Position ausgeben
4  console.log(text.charAt(4)); // i
5  // Prüfen, ob "langer" enthalten ist (gibt true/false)
6  console.log(text.includes("langer")); // true
7  // Substring: Start (inkl.) bis Ende (exkl.)
8  console.log(text.substring(4, 7)); // ist
9  // Slice: Die letzten 4 Zeichen ausschneiden (Negativer Index)
10 console.log(text.slice(-4)); // Text
11 // String bei den Leerzeichen teilen
12 console.log(text.split(" ")); // ["Das", "ist", "ein", "langer", "Text"]
13 // Länge des Strings
14 console.log(text.length); // 23

```

Date

Das Objekt `Date` gewährt Zugriff auf das aktuelle Datum und die Uhrzeit. Wird ein neues `Date`-Objekt mit `new` erzeugt, so wird das zu diesem Zeitpunkt aktuelle Datum (am ausführenden Rechner) darin gekapselt. Intern wird es durch die Anzahl der Millisekunden repräsentiert, die seit dem 1. Jänner 1970 um 00:00:00 Uhr vergangen sind. Das Objekt bietet aber viele Methoden, um einzelne Aspekte dieses Datums abzufragen oder auszugeben:

- `getFullYear()`: gibt das Jahr im Datum zurück,
- `getDay()`: gibt den Wochentag als Zahl (0–6) zurück,
- `getHours()`: gibt die aktuelle Stunde aus, oder
- `getMinutes()`: gibt die aktuelle Minute zurück.
- `toLocaleDateString()`: gibt einen formatierten, sprachsensitiven Datums-String aus.
- `Date.now()`: Gibt den aktuellen Unix-Timestamp (Millisekunden seit 01.01.1970) zurück (statische Methode).

```

1  const d = new Date();
2
3  // Komplettes Datum ausgeben (entspricht d.toString())
4  console.log(d); // z.B. Fri Jan 09 2026 13:29:12 GMT+0100 (Central European Standard
   Time)
5  // Jahr ausgeben
6  console.log(d.getFullYear()); // z.B. 2026
7  // Wochentag ausgeben

```

```

8 console.log(d.getDay()); // z.B. 5 für Freitag
9 // Stunde ausgeben
10 console.log(d.getHours()); //z.B. 13
11 // Minute ausgeben
12 console.log(d.getMinutes()); // z.B. 29
13 // Formatiert für Österreich ausgeben
14 const options = { weekday: "long", year: "numeric", month: "long", day: "numeric" };
15 console.log(d.toLocaleDateString("de-AT", options)); // z.B. Freitag, 9. Januar 2026
16 // Unix-Timestamp
17 console.log(Date.now()); // z.B. 1767961752219

```

Math

Das Objekt `Math` ist ein mathematisches Hilfsobjekt. Es stellt verschiedene Konstanten wie π (`Math.PI`) oder e (`Math.E`) zur Verfügung oder bietet Methoden wie `Math.sin()` oder `Math.cos()`. `Math` ist ein statisches Objekt, es muss daher nicht mit `new` erzeugt werden.

Das Objekt `Math` dient als Container für mathematische Konstanten und Funktionen. Es ist ein statisches Objekt und muss daher – im Gegensatz zu `Date` – nicht mit `new` instanziiert werden. Die wichtigsten Eigenschaften und Methoden für den Alltag sind:

- `Math.PI`: die Kreiszahl π ,
- `Math.round(x)`: rundet eine Zahl kaufmännisch auf die nächste Ganzzahl,
- `Math.ceil(x)` / `Math.floor(x)`: rundet eine Zahl stets auf bzw. ab,
- `Math.random()`: liefert eine Zufallszahl im Intervall $[0, 1[$,
- `Math.max(...args)` / `Math.min(...args)`: ermittelt den größten bzw. kleinsten Wert aus einer Liste von Argumenten.

```

1 // Konstanten
2 console.log(Math.PI); // 3.141592653589793
3 // Runden
4 console.log(Math.round(4.6)); // 5
5 console.log(Math.floor(4.9)); // 4
6 // Zufallszahl zwischen 1 und 10 (Ganzzahl)
7 const randomInt = Math.floor(Math.random() * 10) + 1;
8 console.log(randomInt); // z.B. 2
9 // Maximum ermitteln (Spread-Operator '...' entpackt das Array)
10 const values = [10, 42, 5, 100];
11 console.log(Math.max(...values)); // 100

```

Array

Auch Arrays sind in JavaScript Objekte (vom Typ `Array`). Im Gegensatz zu Java-Arrays sind sie jedoch dynamisch (ihre Größe kann sich jederzeit ändern) und können theoretisch Datentypen mischen.

Die Erzeugung erfolgt fast ausschließlich über die Literal-Schreibweise mit eckigen Klammern. Der Konstruktor `new Array()` existiert zwar, wird aber aufgrund von eigenwilligem Verhalten (bei nur einem Zahl-Argument wird ein leeres Array mit dieser Länge erzeugt) kaum verwendet.

```

1 // Standard: Array-Literal

```

```

2 const numbers = [1, 2, 3];
3 // Ungewöhnlich / Veraltet
4 const empty = new Array(3); // Erzeugt [undefined, undefined, undefined]

```

Das Array-Objekt bietet eine Vielzahl an Methoden zur Manipulation und Iteration. Hier sind die wichtigsten, unterteilt in klassische Manipulation und moderne, funktionale Methoden. Die vollständige Liste findet sich unter [2, Abschnitt 23.1.3] oder bei MDN².

Manipulation (verändert das Original-Array):

- `push(element)`: fügt ein Element am Ende hinzu,
- `pop()`: entfernt das letzte Element und gibt es zurück,
- `splice(start, count)`: entfernt `count` Elemente ab Index `start` (und kann optional neue einfügen),
- `sort()`: sortiert das Array (standardmäßig alphabetisch, auch bei Zahlen).

Iteration und Abfrage (gibt ein neues Array zurück):

- `includes(value)`: prüft, ob `value` im Array enthalten ist (liefert `true/false`),
- `forEach(callback)`: führt eine Funktion für jedes Element aus (Ersatz für `for`-Schleifen),
- `filter(callback)`: erstellt ein *neues* Array mit allen Elementen, für die die Callback-Funktion `true` liefert,
- `map(callback)`: erstellt ein *neues* Array, indem die Callback-Funktion auf jedes Element angewendet wird (Transformation).

```

1 const grades = [1, 2, 3, 4, 5];
2
3 // 1. Element hinzufügen
4 grades.push(1);
5 console.log(grades); // [1, 2, 3, 4, 5, 1]
6 // 2. Filtern (nur positive Noten <= 4)
7 // Die Arrow-Function übergibt jedes Element im Array und gibt nur die zurück, die <= 4 sind
8 const passed = grades.filter(n => n <= 4);
9 console.log(passed); // [1, 2, 3, 4, 1]
10 // 3. Transformieren (Noten verdoppeln – Mapping)
11 const doubled = passed.map(n => n * 2);
12 console.log(doubled); // [2, 4, 6, 8, 2]
13 // 4. Prüfen ob enthalten
14 console.log(grades.includes(5)); // true

```

9.1.6 Alle Eigenschaften eines Objekts ausgeben: `for...in` Schleife

Während die in Abschnitt 8.6.2 behandelte `for...of`-Schleife ideal ist, um über die Werte von Arrays zu iterieren, funktioniert sie nicht direkt auf regulären Objekten (da diese standardmäßig nicht iterierbar sind).

Um ein Objekt zu durchlaufen, wird stattdessen die `for...in`-Schleife verwendet. Sie iteriert nicht über die Werte, sondern über die **Schlüssel** (Eigenschaften) des Objekts.

²https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array#instance_methods

Ebenfalls zu den Kontrollstrukturen aus Abschnitt 8.6 gehört die `for...in`-Schleife. Sie iteriert über ein Objekt (d.h. auch über ein Array) und gibt dabei jeden Eintrag zurück. Die prinzipielle Syntax lautet wie folgt:

```
1 for (Variable in Objekt) {  
2   Anweisung  
3 }
```

Zunächst wird eine Variable definiert, in der bei jedem Schleifendurchlauf der Name der aktuellen Eigenschaft gespeichert wird. Um an den dazugehörigen Wert zu gelangen, muss auf das Objekt mittels der Klammern-Notation (`Objekt[Variable]`) zugegriffen werden. Folgendes Beispiel durchläuft ein vereinfachtes Studierenden-Objekt (ohne Methoden) und gibt jede Eigenschaft in der Konsole aus.

```
1 const student = {  
2   name: "John Doe",  
3   age: 22,  
4   program: "MTD",  
5   languages: ["Java", "C#"]  
6 };  
7  
8 for (const property in student) {  
9   console.log("Eigenschaft:", property);  
10  console.log("Wert:", student[property]);  
11 }
```

Unterschied zu `for...of`

Es ist wichtig, die beiden Schleifentypen nicht zu verwechseln:

- `for...of` iteriert über Werte (ideal für Arrays).
- `for...in` iteriert über Schlüssel (ideal für Objekte).

Wird `for...in` auf ein Array angewandt, werden nicht die Werte, sondern der Index (0, 1 etc.) zurückgegeben. Zudem ist die Reihenfolge bei `for...in` nicht garantiert, weshalb sie für Arrays vermieden werden sollte.

9.1.7 JavaScript-Objekte und JSON

JSON basiert auf JavaScript und ist ein Subset der dortigen Objekt-Literal-Notation, die in Abschnitt 9.1.4 vorgestellt wurde. Somit ist jedes gültige JSON-Dokument auch gültiges JavaScript. Um externe JSON-Daten (die meist als String vorliegen) in ein nutzbares JavaScript-Objekt umzuwandeln, steht das globale Objekt `JSON`³ zur Verfügung. Dessen Methode `parse()`⁴ analysiert den String und erstellt das entsprechende Objekt.

Um auf die Werte eines JSON-Datensatzes in JavaScript zuzugreifen, kommt die übliche Syntax für Objekte und Arrays zum Einsatz. Eigenschaften von Objekten werden über die Punktnotation angesprochen, auf Einträge in Arrays wird mit eckigen Klammern und der Indexposition zugegriffen. Da JSON ein Subset der Objekt-Literal-Notation ist, kann das Tiroler Knödel-Rezept aus Vorlesung 7, Abschnitt 7.2.2 zu Test-

³https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

⁴https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

zwecken (etwas reduziert) als Variable – hier `rezept` – angelegt werden. Die einzelnen Werte werden dann wie bei Objekten und Arrays abgefragt und ausgegeben:

```

1  const rezept = {
2    "gericht": "Der klassische Tiroler Knödel",
3    "zutaten": [
4      {
5        "ingredienz": "Semmelwürfel",
6        "menge": 350,
7        "einheit": "g"
8      },
9      {
10       "ingredienz": "Milch",
11       "menge": 200,
12       "einheit": "ml"
13     }
14   ],
15   "zubereitung": [
16     "Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen lassen.",
17     "Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen Zutaten eine feste, aber nicht zu feuchte Masse kneten."
18   ]
19 };
20
21 // Zugriff auf einfache Eigenschaft
22 console.log(rezept.gericht); // Der klassische Tiroler Knödel
23 // Zugriff auf Array an Index 0, davon Eigenschaft 'ingredienz'
24 console.log(rezept.zutaten[0].ingredienz); // Semmelwürfel
25 // Zugriff auf Array an Index 1, davon Eigenschaft 'menge'
26 console.log(rezept.zutaten[1].menge); // 200
27 // Zugriff auf zweiten Zubereitungsschritt
28 console.log(rezept.zubereitung[1]); // Kleinwürfelig geschnittenen Speck ...

```

9.2 Client-Side JavaScript

Alle bisher besprochenen Dinge betrafen ausschließlich die Sprache JavaScript (d.h. ECMAScript). Bis auf die Teile, wo Output in der Konsole zum besseren Verständnis erzeugt wurde, war jeglicher Code völlig vom Ausgabemedium unabhängig. Sobald JavaScript aber für den Einsatz im Browser konzipiert werden soll, reicht diese Ausgabeunabhängigkeit nicht mehr aus. Es muss Mechanismen geben, wie mit JavaScript generierter Code dargestellt werden kann, oder wie Daten zur Weiterverarbeitung in JavaScript erfasst werden können. Dieser Teil wird *Client-Side JavaScript* genannt – JavaScript für den Einsatz in Webbrowsern.

9.2.1 Allgemeines

JavaScript im Webbrowser sollte dazu verwendet werden, Websites hinsichtlich ihrer Funktionalität zu bereichern. Jedoch sollte eine Seite auch ohne JavaScript immer funktionieren, wenngleich die Funktionalität eingeschränkt sein mag. JavaScript kann von

Benutzer*innen jederzeit deaktiviert werden, auch für Suchmaschinen und ähnliche automatisierte Systeme kann JavaScript-Code nicht vollständig lesbar sein. Daher sollten Webseiten Prinzipien wie *Graceful Degradation* oder *Progressive Enhancement*⁵ berücksichtigen und auch dann noch funktionieren, wenn das Script nicht verfügbar ist. Ist dies nicht möglich, so sollte zumindest das Element `<noscript>` verwendet werden, in dem Benutzer*innen darauf hingewiesen werden können, dass die Seite sehr stark auf JavaScript basiert und dieses zur Zeit nicht verfügbar ist. Der Inhalt dieses Elements wird nur dann angezeigt, wenn JavaScript nicht aktiviert ist.

Um nun JavaScript im Webbrowser verwenden zu können, stellt dieser eine Umgebung zur Verfügung, mit der spezifisch auf das Browserfenster und die darin angezeigte Website zugegriffen werden kann. Diese Zugriffsmöglichkeiten sind:

- das `window`-Objekt für den Zugriff auf das Browserfenster (siehe Abschnitt 9.2.3),
- das `document`-Objekt (mit dem Document Object Model) für den Zugriff auf die aktuelle Website (siehe Abschnitt 9.2.4 und Vorlesung 10).
- ein event-basiertes Programmiermodell, welches es erlaubt, auf Eingaben der Benutzer*innen zu reagieren (siehe Vorlesung 10).

Bevor auf diese Dinge genauer eingegangen wird, ist zunächst die prinzipielle Einbindung von JavaScript in HTML wichtig.

9.2.2 Einbindung in HTML

Um JavaScript im Zusammenhang mit HTML verwenden zu können, ist eine *Einbindung* ins HTML-Markup nötig. Wie bereits bei CSS gibt es auch hier mehrere Möglichkeiten:

- direkt im HTML-Markup (Inline),
- Einbindung als externe JavaScript-Datei,
- Verwendung mit Event-Handlern.

Inline-Einbindung

Um Scripts in HTML einbinden zu können, steht das Element `<script>` (geschlossen durch `</script>`) zur Verfügung. Optional ist die Angabe des Attributs `type`, welches im Falle von JavaScript den Wert `text/javascript` haben muss. Mit HTML5 wurde dieser Typ jedoch als Standard für das `<script>`-Element festgelegt, sodass die Angabe nicht benötigt wird.

Da Browser HTML-Dokumente zeilenweise von oben nach unten abarbeiten, ist die Position des Skripts entscheidend: Greift das Skript auf HTML-Elemente zu (wie im nachfolgenden Beispiel auf das `<div>`-Element), müssen diese Elemente zum Zeitpunkt der Ausführung bereits existieren.

Bei direkter Einbindung platziert man das `<script>`-Element daher klassischerweise am Ende des `<body>`-Bereichs, direkt vor dem schließenden `</body>`-Tag. So ist sichergestellt, dass der gesamte HTML-Inhalt geladen ist, bevor das Skript läuft.

Es können beliebig viele `<script>`-Elemente zum Einsatz kommen, sie alle zählen als ein großer JavaScript-Kontext, d. h. eine globale Variable gilt in allen Bereichen derselben HTML-Datei.

⁵https://www.w3.org/wiki/Graceful_degradation_versus_progressive_enhancement

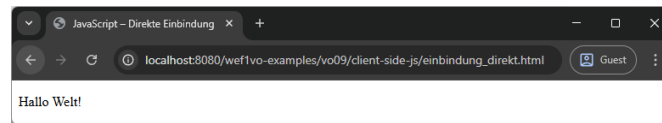


Abbildung 9.1: Der Text wird mittels JavaScript dynamisch in das `<div>`-Element eingefügt.

Das folgende Beispiel erzeugt seinen Output nur über JavaScript. Im `<body>` liegt ein leeres `<div>`-Element mit der ID `output`, der `<script>`-Tag ist danach platziert. Über das `document`-Objekt (siehe Abschnitt 9.2.4) wird mit der Methode `getElementById()` das `<div>` abgefragt. Über die Eigenschaft `innerHTML` wird ein HTML-Fragment – in diesem Fall ein Absatz mit Text – in das Element gesetzt. Abbildung 9.1 stellt das Ergebnis dar.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript &ndash; Inline-Einbindung</title>
6   </head>
7   <body>
8     <div id="output"></div>
9     <script>
10       document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";
11     </script>
12   </body>
13 </html>
```

Bei einem Blick in den Quellcode, sind lediglich das leere `<div>` und die JavaScript-Anweisung ersichtlich, nicht etwa der von JavaScript erzeugte HTML-Code. Die Dev-Tools im Browser können allerdings auch generierten Code anzeigen, also wie die HTML-Struktur aussieht, nachdem JavaScript abgearbeitet wurde.

Externe Einbindung

Die wesentlich elegantere Methode ist die Einbindung über eine *externe* JavaScript-Datei. JavaScript-Dateien sind normale Textdateien und haben die Endung `.js`. Sie werden beim `<script>`-Element mit dem `src`-Attribut eingebunden. Wichtig ist, dass das `<script>`-Element trotzdem mit `</script>` abgeschlossen wird, auch wenn es durch die externe Einbindung keinen Inhalt mehr hat.

Bei extern eingebundenen JavaScript-Dateien gibt es mehr Möglichkeiten für die Platzung des `<script>`-Elements.

1. **Im `<head>` mit dem Attribut `defer` (empfohlen):** Das Skript wird parallel zum HTML heruntergeladen, aber erst ausgeführt, wenn das HTML vollständig geladen ist. Somit funktionieren auch Skripte, die auf die HTML-Struktur zugreifen, denn obwohl das Skript zuerst im Quellcode an die Reihe kommt, wartet der Browser aufgrund von `defer` mit dessen Abarbeitung bis die Seitenstruktur fertig geladen ist. Dies ist aktuell die beste Lösung.
2. **Im `<head>` mit `async`:** Das Skript wird ebenfalls parallel zur HTML-Verarbeitung

geladen, jedoch *sofort* nach dem fertigen Download ausgeführt (wobei der HTML-Parser kurzzeitig pausiert). Dies eignet sich daher nur für unabhängige Skripte, die nicht auf Elemente der Seite, die erst später geladen werden, zugreifen und ist im Kontext dieser Lehrveranstaltung daher nur kaum zu gebrauchen.

3. **Am Ende des <body>:** Früher war dies der Standard, um das Blockieren des Seitenaufbaus zu verhindern. Das Skript wird erst geladen, wenn der Browser am Ende der Seite ankommt. Dadurch ist zwar garantiert, dass die HTML-Struktur geladen und bekannt ist, allerdings kann das Laden der JavaScript-Datei erst danach erfolgt, während mit **defer** diese schon parallel geladen wird, weshalb Option 1 zu bevorzugen ist.
4. **Ohne Attribute im <head>:** Dies sollte vermieden werden, da das Skript das Laden der restlichen Seite blockiert, bis es vollständig heruntergeladen und ausgeführt wurde. Außerdem sind die Elemente der Seite im <body> noch nicht verfügbar, da das Skript vor deren Parsen ausgeführt wird.

Das folgende Markup zeigt das obige Beispiel mit externer Einbindung im <head> mit **defer**-Attribut.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>JavaScript &ndash; Externe Einbindung</title>
6     <script src="extern.js" defer</script>
7   </head>
8   <body>
9     <div id="output"></div>
10  </body>
11 </html>

```

Der JavaScript-Code wurde in eine eigene Datei (**extern.js**) ausgelagert. Das Ergebnis gleicht dem in Abbildung 9.1.

```

1 document.getElementById("output").innerHTML = "<p>Hallo Welt!</p>";

```

Einbindung über Event-Handler

JavaScript stellt so genannte *Event-Handler* zur Verfügung, mit denen auf Eingaben von Benutzer*innen reagiert werden kann. Code wird hier nur ausgeführt, wenn ein bestimmtes Ereignis (z. B. ein Klick) eintritt. Diese Event Handler haben Namen wie **onclick**, **onmousedown** etc. und werden im einfachsten Fall als HTML-Attribute angeführt und können in ihrem Wert JavaScript-Code aufrufen.

Folgender Code erzeugt beim Klick auf einen generischen Button (**type="button"**) eine Dialogbox mit Text, wenn darauf geklickt wird.

```

1 <button type="button" onclick="alert('Hallo Welt!')">Klick mich!</button>

```

Die Definition solcher Event-Handler ist zwar für einfache Beispiele nützlich, in komplexeren Anwendungen werden diese jedoch meist direkt in JavaScript registriert. Mehr Details dazu folgen in Vorlesung 10.

9.2.3 window-Objekt

Das **window**-Objekt wird vom Browser automatisch zur Verfügung gestellt und stellt die Grundlage dar, um JavaScript-Code im Browser ausführen zu können. Es wird auch als *globale Objekt* (engl. „global object“) bezeichnet und definiert den gesamten Ausführungskontext. Es bietet vorrangig Zugriff auf das Browserfenster und seine Eigenschaften und erlaubt es, diese sowohl auszulesen, als auch neu zu setzen. Im Folgenden werden die Eigenschaften des **window**-Objekts – auch bezeichnet als das Browser Object Model (BOM) – überblicksartig beschrieben. Eine vollständige Auflistung bietet wie immer MDN⁶.

Eigenschaften

Das **window**-Objekt selbst bietet unter anderem folgende Eigenschaften. Mit ihnen können Details zum Browserfenster abgefragt und teilweise auch gesetzt werden. Unter den Eigenschaften sind auch wiederum einige Objekte, die Detailinformationen über bestimmte Dinge (wie etwa die aktuelle Seite) beinhalten. Der Zugriff erfolgt über Punktnotation (z. B. **window.document**, der Objektname **window** kann aber auch weggelassen werden, wenn das aktuelle Fenster bearbeitet wird (also nur noch **document**)).

- **document**: das aktuell angezeigte HTML-Dokument (Objekt),
- **history**: die aktuelle Browser-History (Objekt). Die letzte aufgerufene Seite kann etwa über den Aufruf von **history.back()** erreicht werden,
- **location**: Zugriff auf den vollständigen URL der Seite (Objekt),
- **screen**: Informationen über den Bildschirm, wie etwa Höhe, Breite oder Farbtiefe (Objekt),
- **innerWidth**: die Breite des Anzeigebereichs im Browserfenster,
- **innerHeight**: die Höhe des Anzeigebereichs im Browserfenster.

Methoden

Das **window**-Objekt bietet auch Methoden an, die vornehmlich zur Erzeugung und Veränderung von Browserfenstern eingesetzt werden können:

- **open(url, name, features)**: Öffnet ein neues Browserfenster und lädt die Seite an der Adresse **url**, gibt dem Fenster den Namen **name** (um es später damit in JavaScript wieder ansprechen zu können) und stellt die Eigenschaften **features** im neuen Fenster bereit (z. B. **width**, **height**). Moderne Browser verhindern dies aber, wenn es nicht explizit durch direkten Klick ausgelöst wird.)
- **close()**: schließt ein Fenster (funktioniert in der Regel aber nur, wenn das Fenster auch per Script geöffnet wurde),
- **alert(text)**: zeigt eine Dialogbox mit dem Inhalt **text** an,
- **scrollBy(x, y)**: scrollt im Fenster um eine durch **x** und **y** spezifizierte Anzahl von Pixeln,
- **focus()**: schaltet das aktuelle Fenster aktiv (holt es in den Vordergrund).

⁶<https://developer.mozilla.org/en-US/docs/Web/API/Window>



Abbildung 9.2: Zunächst wird ein neues Fenster auf Klick geöffnet. Solange es offen ist, zeigt das Dialogfenster diesen Umstand korrekt an.

Beispiel: Fenster öffnen

Das folgende Beispiel öffnet ein Fenster mit JavaScript und stellt eine Funktion zur Verfügung, die überprüft, ob das Fenster geöffnet oder geschlossen wurde. Ebenso kann das Fenster wieder geschlossen werden. Der JavaScript-Code wird über eine externe Datei eingebunden. Es werden Buttons verwendet, deren `onclick`-Event die Funktionen auslösen. Abbildung 9.2 zeigt das Ergebnis im Browser.

```

1 <html lang="de">
2   <head>
3     <meta charset="utf-8">
4     <title>window-Objekt &ndash; Beispiel</title>
5     <script src="window-demo.js" defer></script>
6   </head>
7   <body>
8     <button type="button" onclick="openWindow()">Öffne Fenster</button>
9     <button type="button" onclick="closeWindow()">Schließe Fenster</button>
10    <button type="button" onclick="checkOpen()">Überprüfe Fenster</button>
11  </body>
12 </html>

```

Der JavaScript-Code sieht wie folgt aus:

```

1 let myWindow = null;
2
3 function openWindow() { // Öffnet ein kleines Popup-Fenster
4   myWindow = window.open("window.html", "NewWindow", "width=300, height=200");
5   if (myWindow) {
6     myWindow.focus();
7   }
8 }
9
10 function closeWindow() { // Schließt das Popup-Fenster
11   if (myWindow && !myWindow.closed) {
12     myWindow.close();
13   }
14 }
15
16 function checkOpen() { // Gibt den Status des Popup-Fensters aus
17   alert(myWindow && !myWindow.closed ? "Fenster ist offen!" : "Fenster ist
    geschlossen!");
18 }

```

9.2.4 document-Objekt

Das `document`-Objekt ist ein Unterobjekt des `window`-Objekts und bietet umfassenden Zugriff auf das zurzeit im Browser geöffnete Dokument. Auch es besitzt eine Reihe von Eigenschaften und Methoden, mit denen das aktuelle Dokument verändert werden kann. Es folgen die wichtigsten dieser Punkte im Überblick, eine vollständige Auflistung bietet MDN⁷.

Eigenschaften

Folgende beispielhafte Eigenschaften können etwa vom aktuellen Dokument ausgelesen bzw. teilweise auch gesetzt werden:

- `title`: der Titel des Dokuments, wie mittels `<title>` spezifiziert,
- `URL`: der URL des aktuellen Dokuments,
- `referrer`: die zuvor besuchte Seite (von der auf die aktuelle verlinkt wurde),
- `lastModified`: das Datum der letzten Änderung am Dokument.

Methoden

Während früher Methoden wie `document.write()` genutzt wurden, um Text direkt in den Datenstrom der Seite zu schreiben, gilt dies heute als veraltet und sollte nicht mehr verwendet werden. Das `document`-Objekt dient heute primär als Einstiegspunkt in das DOM, um Elemente zu finden. Hier zwei Beispiele, mehr dann in Vorlesung 10.

- `getElementById(id)`: Findet ein HTML-Element anhand seiner ID.
- `querySelector(selector)`: Findet das erste Element, das auf einen CSS-Selektor passt.

Beispiel: Dokument-Eigenschaften nutzen

Das folgende Beispiel ruft auf einen Klick eine JavaScript-Funktion auf, die Informationen über das aktuelle Dokument ausgibt und den Titel der Seite dynamisch ändert. Abbildung 9.3 zeigt die Seite vor und nach dem Klick.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>Dokument-Eigenschaften auslesen</title>
6     <script src="document-demo.js" defer></script>
7   </head>
8   <body>
9     <h1>Dokument-Manipulation</h1>
10    <p>DevTools öffnen (F12) und auf den "Console"-Tab wechseln, um die Ausgabe des
      Beispiels zu sehen.</p>
11    <button type="button" onclick="showInfo()">Infos anzeigen & Titel ändern</button>
12  </body>
13 </html>
```

⁷<https://developer.mozilla.org/en-US/docs/Web/API/Document>

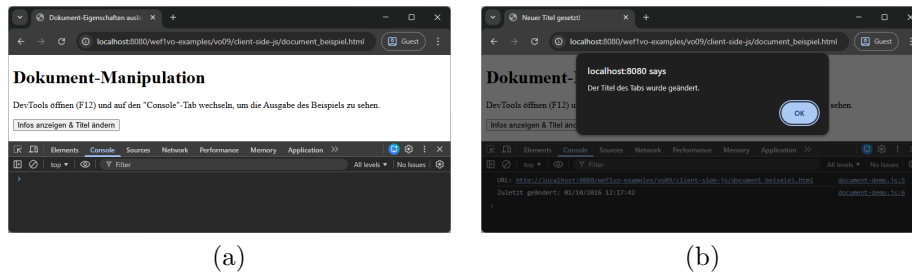


Abbildung 9.3: Das „alte“ Dokument (a) ist der Ursprungszustand. Der Titel lautet „Dokument-Eigenschaften auslesen“, die Console ist leer. Wird auf den Link geklickt, wird die Funktion `showInfo()` aufgerufen, die Informationen in die Konsole schreibt und den Titel der Seite auf „Neuer Titel gesetzt!“ ändert (b).

In der JavaScript-Datei wird `showInfo()` definiert:

```
1 function showInfo() {
2   // Zugriff auf Eigenschaften
3   console.log("URL: " + document.URL);
4   console.log("Zuletzt geändert: " + document.lastModified);
5
6   // Ändern einer Eigenschaft
7   document.title = "Neuer Titel gesetzt!";
8   alert("Der Titel des Tabs wurde geändert.");
9 }
```

Document Object Model

Die wichtigste Aufgabe des `document`-Objekts ist jedoch wie schon kurz erwähnt das Zurverfügungstellen des Document Object Models (kurz DOM). Dieses erlaubt wahlfreien Zugriff auf alle Elemente des aktuellen HTML-Dokuments. Dieses Thema wird das Hauptaugenmerk von Vorlesung 10 und wurde daher an dieser Stelle nur kurz vorab erwähnt.

9.3 Weiterführende Literatur

Die Literaturempfehlungen gleichen prinzipiell denen von Vorlesung 8. Empfohlen für die Arbeit mit Objekten und die Einbindung im Browser werden insbesondere [1, 3, 4] empfohlen. Sie gehen im Detail auf diese Konzepte ein und warten mit vielen Beispielen auf.

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 9-20).
- [2] European Computer Manufacturers Association. *ECMAScript[®] 2025 Language Specification*. ECMA-262. Version 16.0. Juni 2025. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-262/> (siehe S. 9-7, 9-11).
- [3] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 9-20).
- [4] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 9-20).